

가시화 요구사항 쉽게 설명하기

엑셀 시트의 VZ-* 28개 요구사항을
말로 설명할 수 있는 형태로 푼 문서

가시화는 데이터를 직접 만들지 않습니다.
받아서 그리고, 사용자가 누른 걸 백엔드에 넘깁니다.

작성 김현우 (가시화 담당)

작성일 2026-08-20

기준 피지컬팀 프로젝트 mk2 요구사항 정의서 · 김현우 시트

구성 수신 10 · 송신 5 · 화면 7 · LLM/음성 4 · 공통 2

가시화 요구사항 — 쉽게 설명하기

엑셀 시트의 VZ-* 28개를 말로 설명할 수 있는 형태로 쓴 문서. 각 항목은 ① 한 줄로 말하면 ② 쉽게 풀면 ③ 왜 이 주기인가 ④ 안 하면 어떻게 되나 순서.

0. 전체를 한 문장으로

"가시화는 데이터를 직접 만들지 않습니다. 받아서 그리고, 사용자가 누른 걸 백엔드에 넘깁니다."

여기서 두 가지가 따라 나옵니다.

첫째, 브라우저는 아무 데도 직접 붙지 않습니다. 문이 딱 네 개뿐이에요 — 실시간은 **WS 게이트웨이**, 지표는 **질의 프록시**, 감사는 **조회 API**, 구성은 **레지스트리 조회**. 브로커나 DB에 직접 접속하지 않습니다. 이유는 단순해요. 브라우저는 사용자 손 안에 있어서 열쇠를 쥐여주면 안 됩니다.

둘째, 판단은 다른 파트가 하고 저는 표시합니다. 좌표 변환은 백엔드, 명령 확정 판정도 백엔드, 음성 의도 해석과 계획 생성은 AI. 제가 하는 건 **받은 걸 사람이 알아볼 수 있게 만드는 것과 사람이 한 행동을 정확히 넘기는 것**입니다.

주기를 정할 때 쓴 원칙도 하나입니다 — **"원천이 주는 것보다 자주 그려봐야 새 그림이 없다."** 그래서 대부분의 숫자는 하드웨어·AI가 정한 주기에서 따왔습니다.

구성 — 수신 10 · 송신 5 · 화면 7 · LLM/음성 4 · 공통 2 = **28개**

수신 (VZ-I) — 받아오는 것 10개

VZ-I-01 · 실시간 상태 구독

한 줄로 말하면

"로봇·센서 상태를 백엔드를 거쳐 실시간으로 구독합니다. 상태는 한 덩어리가 아니라 세 층으로 받습니다."

쉽게 풀면 브라우저는 장치가 쓰는 전송 프로토콜에 직접 붙을 수 없습니다. 그래서 백엔드가 중간에서 받아 웹소켓으로 넘겨줍니다. 구독할 때 **채널 주소가 아니라 "어느 개체, 어느 노드, 어떤 채널"**로 요청합니다. 전송 규격이 바뀌어도 제 화면이 안 깨지게요.

왜 이 주기인가 로봇은 임무 중 50ms(초당 20번)로 상태를 올립니다. 그런데 **올 때마다 화면을 다시 그리면 오히려 끊깁니다.** 그래서 데이터는 다 받되 **화면 반영만 0.1초 단위로 묶어서** 합니다.

다만 원천마다 속도가 다릅니다. 로봇은 50ms로 촘촘한데, **센서는 평시 1분에 한 번이고 임계값을 넘거나 값이 급변할 때만 즉시** 올라옵니다. 그래서 평소엔 한산하다가 이벤트 때 몰리는 패턴이에요.

안 하면 초당 20번 렌더링하다가 브라우저가 버벅입니다. 반대로 데이터를 버리면 이동 궤적이 끊겨 보이고요.

VZ-I-02 · 재접속 시 현재값 확보

한 줄로 말하면

"화면을 새로 열면 그 순간 현재 상태를 한 번 받아야 합니다. 안 그러면 빈 화면입니다."

쉽게 풀면 발행-구독 방식은 "지금부터 오는 것"만 줍니다. 과거 값은 안 줘요. 그러니 화면을 새로 열면 **다음 데이터가 올 때까지 아무것도 못 그림니다.**

얼마나 비어 있나 — 이게 설득 포인트입니다. - 대기 중인 로봇은 5초마다 보고 → 최대 5초 공백 - 평시 센서는 1분마다 보고 → **최대 1분 빈 화면**

해결책은 둘 중 하나 하드웨어가 마지막 값을 보존하거나, 백엔드가 들고 있다가 구독하는 순간 한 번 던져주거나. **어느 쪽이든 상관없습니다.** 다만 정해져야 합니다.

VZ-I-03 · 구성(레지스트리) 조회

한 줄로 말하면

"'지금 뭐가 있어야 하는지' 명단을 받아야 합니다. 반 명부 같은 거예요."

쉽게 풀면 출석부가 없으면 **결석한 학생을 표시할 수 없습니다.** 온 학생만 보이니까요. 마찬가지로 배포되지 않은 로봇은 데이터를 안 보내니 상태 메시지만으로는 **화면에 영원히 안 나타납니다.**

받아야 하는 것 — 있어야 할 개체 목록, 각 개체가 어느 구역 소속인지, 구역 원점이 전역 어디인지, 그리고 표시 이름·별칭.

왜 이 주기인가 명단은 장치를 등록하거나 배포하거나 구역을 옮길 때만 바뀝니다. **처음에 한 번 + 바뀌면 알려주기**면 충분합니다.

안 하면 "의도적으로 안 켜진 것"과 "고장 나서 안 오는 것"을 구분할 수 없습니다. **구현 자체가 불가능**해집니다.

VZ-I-04 · 지표 질의

한 줄로 말하면

"그래프용 과거 데이터는 백엔드 프록시를 통해 조회합니다."

왜 이 주기인가 — 숫자 근거가 제일 깔끔합니다. 하드웨어 관측 지표는 **15초에 한 번 수집**됩니다. 그러면 제가 1초마다 물어봐도 **대부분 아까 그 값**이에요. 그래서 기본 15초로 맞췄습니다.

단, 센서가 이벤트 모드로 들어가면 보고 주기가 1초로 올라갑니다. 홍수 대응 골든타임이니깐요. 그때는 저도 1초로 따라 갑니다.

조회 범위가 길면(1시간 이상) 점 간격이 넓어져서 30초로 늦춥니다.

VZ-I-05 · 감사 이력 조회

한 줄로 말하면

"'이 장비를 마지막으로 누가 조작했나'와 그 판단 근거를 화면에 띄우려면 감사 기록을 되읽어야 합니다."

쉽게 풀면 제가 감사를 **쓰지는 않지만 읽기는 해야** 합니다. 쓰기만 있고 읽기가 없으면 책임 추적이 화면에서 확인되지 않아요.

왜 이 주기인가 감사는 이미 지나간 기록입니다. **패널을 열 때만** 조회합니다. 진행 중인 명령의 상태 변화는 결과 푸시로 따로 오니까 중복이고요.

VZ-I-06 · 영상 스트림 표시

한 줄로 말하면

"카메라 영상은 브라우저가 재생할 수 있는 형태로 바뀌서 받아야 합니다."

쉽게 풀면 — 영상이 세 갈래입니다

무엇	용도	규격
로봇 관제용 영상	제 화면에 띄우는 것	온디맨드 15fps 고해상도
고정 비전센서 영상	트윈 맵 갱신 + 제 화면	상시 15fps
로봇 인식용 프레임	AI 추론 입력	다운스케일(640급)

세 번째는 **AI가 먹는 것이라 제 화면과 무관**합니다. 원래 하나로 뭉쳐 있던 걸 하드웨어가 분리해줬어요.

카메라 원본 규격은 브라우저에서 그냥 안 들어집니다. 어딘가에서 변환해야 하는데, 그 지점이 아직 안 정해졌습니다.

왜 이 주기인가 원본이 15fps니 **그 이상 요구해봐야 받을 게 없습니다**. 그리고 카메라 전부를 상시로 들면 무선 대역폭과 브라우저 디코딩을 동시에 낭비하니, **열어놓은 패널만** 받습니다.

VZ-I-07 · 탐지 결과 오버레이 정합

한 줄로 말하면

"AI가 찾은 물체 박스를 영상 위에 겹치려면, 그 박스가 몇 번째 사진 것인지 알아야 합니다."

쉽게 풀면 사진에 번호를 붙여 보내고, AI가 결과를 줄 때 **그 번호를 그대로 돌려주는** 겁니다. 그래야 "4821번 사진의 결과"를 4821번 사진 위에 그릴 수 있어요.

안 하면 온디바이스 추론이 0.2초 걸리고 영상이 15fps니까, 결과가 돌아왔을 때 화면은 이미 **3프레임 앞서 있습니다**. 번호 없이 도착 순서대로 그리면 박스가 항상 **사람이 지나간 자리에** 남아요.

여기에 하나 더 — bbox 좌표가 픽셀인지 0~1 비율인지, 원점이 어디인지, 기준 해상도가 뭔지도 정해져야 합니다.

지금 걸린 문제 AI와 하드웨어의 프레임 참조 형식이 다릅니다. **이게 통일 안 되면 이 기능 자체가 성립하지 않습니다**.

VZ-I-08 · 위험도 상태 표시 (신설)

한 줄로 말하면

"지금이 평시인지 관찰인지 경보인지, 그리고 왜 그렇게 판단했는지를 화면에 보여줍니다."

쉽게 풀면 AI가 강우·수위 같은 신호를 보고 **평시 / 관찰 / 경보 / 복구** 네 단계로 상황을 판정합니다. 여기에 위험도 점수와 판단 근거, 권고 조치가 함께 옵니다.

중요한 건 근거예요. "위험합니다"만 쓰면 관제사가 판단할 수 없습니다. **무엇 때문에 그렇게 봤는지**가 같이 와야 하고, 그게 제어의 근거가 됐다면 제어 이력에서도 되짚을 수 있어야 합니다.

왜 이 주기인가 평시엔 위험도가 거의 안 변해서 계속 물어볼 이유가 없습니다. 대신 **상태가 올라가는 순간은 화면 전환·경보를 유발**하므로 즉시 도달해야 합니다.

VZ-I-09 · 객체 추적 및 궤적 표시 (신설)

한 줄로 말하면

"물체를 한 장면씩 보는 게 아니라, 같은 물체로 이어 붙여 궤적까지 보여줍니다."

쉽게 풀면 탐지 박스만 있으면 매 프레임 **처음 보는 물체**입니다. 추적 식별자가 붙어야 "아까 그 사람이 이쪽으로 왔다"가 되고, 속도·이동 방향·지나온 경로를 그릴 수 있어요.

여기에 두 가지가 더 붙습니다. - **카메라 간 연결** — 여러 카메라에 걸친 같은 물체를 하나로 봅니다. 확신이 낮은 연결은 **불확실하다고 표시**해야 하고요. - **불확실성 표시** — 분류 신뢰도를 화면에서 구분해 보여줍니다. 확실한 것과 애매한 것이 똑같이 보이면 안 됩니다.

단기 궤적 **예측**은 선택적 확장으로 뒀습니다.

왜 이 주기인가 추적은 탐지에서 파생되니 **원천보다 자주 받을 수 없습니다**. 따로 주기를 두면 궤적이 실제 위치와 어긋나고요.

VZ-I-10 · AI 실패 이벤트 알림 (신설)

한 줄로 말하면

"AI가 실패했을 때는 지표가 아니라 알림으로 즉시 받습니다."

쉽게 풀면 정상일 때의 성능 지표(VZ-I-04)와 **실패는 성격이 다릅니다**. 실패는 드물게 일어나지만 일어난 순간이 중요해요. 무엇이 어느 컴포넌트에서 어떤 모델 버전으로 무슨 입력에서 실패했는지가 함께 옵니다.

왜 별도 경로인가 지표 질의는 15초마다 물어봅니다. 실패를 여기 실으면 **최대 15초 늦게 알게 됩니다**. 그래서 도착 즉시 알리는 경로로 분리했습니다.

송신 (VZ-O) — 내보내는 것 5개

VZ-O-01 · 제어 명령 발행

한 줄로 말하면

"사용자 조작을 '무엇을 하라'는 추상 명령으로 백엔드에 넘깁니다. 장비 명령으로 바꾸는 건 백엔드가 합니다."

쉽게 풀면 저는 **수문 개방** 까지만 보냅니다. 그게 실제 장비 명령이 되는 건 백엔드가 어휘집을 보고 번역해요. **장비가 바뀌어도 제 화면은 안 고쳐도 됩니다**.

같이 실어 보내는 두 가지 — - **명령 식별자**: 주문번호 같은 것. 이 명령과 그 결과를 짝지어줍니다. - **만료 시각**: 유효기한. 시간이 지난 명령은 실행하면 안 됩니다.

왜 이 주기인가 사용자가 누를 때 나가는 거라 주기가 없습니다. 모아서 보내면 긴급 제어가 늦어지고, 만료 시각의 의미도 사라지고요.

VZ-O-02 · 명령 결과 표시

한 줄로 말하면

"버튼을 눌렀다고 바로 '열림'으로 바꾸지 않습니다. 실제로 열렸다는 확인이 와야 바꿉니다."

쉽게 풀면 — 네 단계로 옵니다

1. **수신 확인** — "명령 받았다"
2. **수행 중** — 물리적으로 움직이는 동안 **200ms마다 진행 상태**가 옵니다
3. **물리 상태 변화** — 실제로 상태가 바뀌었다
4. **완료 / 실패**

화면에는 **진행중 · 확정 · 실패** 세 가지로 보여주되, 2번 덕분에 **움직이는 동안 멈춘 것처럼 보이지 않게** 진행 상태를 함께 표시합니다. 수문처럼 되돌리기 어려운 건 1번이 아니라 3~4번까지 기다려야 합니다.

안 하면 버튼을 누르는 순간 열린 걸로 표시했는데 실제로는 실패 — 화면과 현실이 어긋납니다. 관제 화면에서 제일 위험한 상황입니다.

VZ-O-03 · 책임소재(감사) 필드 전달

한 줄로 말하면

"누가·어떻게 시켰는지를 명령에 같이 실어 보냅니다. 기록을 쓰는 건 백엔드입니다."

쉽게 풀면 — 왜 제가 직접 안 쓰나 세 가지 이유가 있습니다.

1. **위조** — 브라우저가 직접 쓰면 "누가 했다"를 클라이언트가 자기 입으로 말하는 셈입니다. 남의 이름을 넣어도 서버는 받아 적어요.
2. **시계** — 장치들은 NTP로 시각을 맞추니다. 그런데 **브라우저에는 그런 보장이 없어요**. 사용자 PC 시계로 기록하면 법적 추적성을 주장하기 어렵습니다.
3. **유실** — 명령은 실행됐는데 탭을 닫아서 기록이 안 남을 수 있습니다. **감사에서는 최악의 실패**입니다.

그래서 명령과 **한 요청에 같이** 실어 보내고, 조작자와 시각은 백엔드가 토큰과 서버 시계에서 채웁니다.

지금 걸린 문제 음성으로 말했고 LLM이 해석한 경우, 현재 어휘로는 둘 중 **하나만 골라야 합니다**. 어느 쪽을 버려도 사고 원인(잘못 들었나 / 잘못 해석했나)을 구분할 수 없어서, **입력 수단과 판단 주체를 따로 기록**하자고 요청 중입니다.

VZ-O-04 · 자체 관측 지표 발행

한 줄로 말하면

"제 화면이 얼마나 느린지는 제가 직접 관측 시스템에 보냅니다."

쉽게 풀면 다른 데이터는 백엔드를 거치지만 **자기 성능 지표만은 직접** 보냅니다. 남을 관찰한 게 아니라 자기 자신에 대한 값이니깐요. 시도 똑같이 합니다.

왜 60초인가 추세만 보면 되는 값이라 촌촌할 이유가 없습니다. 1초마다 보내면 저장 비용만 늘고 판단은 안 달라져요.

VZ-O-05 · 제어 잠금 상태 표시 (신설)

한 줄로 말하면

"통신이 끊긴 장비는 제어 버튼을 잠그고, 잠겼다는 걸 화면에 보여줍니다."

쉽게 풀면 장비와 연결이 끊기면 **원격 제어가 막힙니다**. 안전 상태를 유지하고, 통신이 돌아와도 **실제 상태를 다시 확인하기 전까지는** 제어를 열지 않아요.

문제는 그 사실을 화면이 모르면 **관제사가 눌러도 실행되지 않는 버튼을 계속 누른다**는 겁니다. 그래서 잠금 상태와 사유를 표시합니다.

왜 이 주기인가 잠금은 통신 두절 판정(하트비트 3회 연속 미수신, 약 3초)과 복구 시점에만 바뀝니다. 대신 **늦게 알면 곤란해서** 전환되는 즉시 받습니다.

화면 (VZ-U) — 그리는 것 7개

VZ-U-01 · 상태 4종 구분 표시

한 줄로 말하면

"장비 상태를 네 가지로 구분합니다 — 정상 / 장애 / 일부러 안 켜 것 / 판단 불가."

쉽게 풀면 — 왜 세 층으로 받아 상태를 아는 주체가 셋입니다.

누가 아는가	무엇을 아는가
기기 자신	"나 고장났어"
서버	"재한테서 연락이 없네"
배포 시스템	"재는 아직 안 켜어"

이걸 하나로 합치면 "**연결은 멀쩡한데 기기가 고장난 상태**"를 표현할 수 없습니다. 연결됐으니 정상으로 보이거든요.

판단 불가가 뭔가 전원은 들어와 있는데 **시계 바늘이 안 움직이는** 상태입니다. 연결은 살아 있는데 값이 갱신되지 않는 거예요.

액추에이터는 어휘가 다릅니다 수문·펌프는 대기 / 동작 중 / 완료 / 오류 / 확인 불가처럼 **자기만의 상태**를 갖습니다. 이걸 표준 3층과 별개인 도메인 어휘로 다룹니다. 공통 컴포넌트가 하드코딩하면 안 돼요.

VZ-U-02 · 디지털 트윈 위치 렌더링

한 줄로 말하면

"로봇 위치를 3D 화면에 놓습니다. 좌표 변환은 백엔드가 하고 저는 배치만 합니다."

쉽게 풀면 로봇은 자기 구역 기준으로 좌표를 말합니다. "여기서 2m 앞"처럼요. 그걸 전체 지도 좌표로 바꾸는 건 백엔드가 합니다. 저는 변환 코드를 **아예 안 만듭니다**.

왜 보간을 하나 50ms마다 오는 좌표를 그대로 찍으면 로봇이 **초당 20번 순간이동**하는 것처럼 보입니다. 서버에 더 자주 보내달라고 하면 무선이 못 버티고요. 그래서 **받은 점 사이를 제가 채워서** 부드럽게 만듭니다.

지금 걸린 문제 좌표가 어느 기준인지(구역 프레임), 어떻게 얻은 값인지(자기추정 / 앵커 보정 / AI 인지)를 알려주는 필드가 **하드웨어 시트에서 빠져 있습니다**. 통합 문서에는 확정으로 들어가 있는데 시트에는 없어요. AI 쪽도 같은 걸 지적했습니다. 확인이 필요합니다.

VZ-U-03 · 추상화 계층 부

한 줄로 말하면

"같은 대상을 결심자·운영자·개발자 시점으로 바꿔 봅니다."

쉽게 풀면 지휘관은 빨강/초록만 보면 되고, 운영자는 어느 장비가 문제인지, 개발자는 원본 수치를 봐야 합니다. **같은 데이터를 세 깊이로** 보여주는 겁니다.

헷갈리기 쉬운 점 구역을 좁혀 들어가는 것(전체 → 구역 → 로봇)과는 **다른 축**입니다. 같은 로봇을 보면서도 판단 수준을 바꿀 수 있어야 해요.

주의할 것 AI가 준 위험도 같은 값은 **이미 요약된 값**입니다. 그걸 원본인 줄 알고 또 평균 내면 틀린 숫자가 나와요.

VZ-U-04 · 노드 편집기 구성 반영

한 줄로 말하면

"화면 구성을 블록 연결로 만들고, '반영' 버튼을 눌러야 실제 화면에 적용됩니다."

쉽게 풀면 데이터를 어떻게 해석해 보여줄지를 **코드로 미리 고정하지 않고** 쓰는 시점에 조합합니다. 편집 중에 자동 반영되면 미완성 구성이 운영 화면에 나가버리니, **명시적으로 눌러야** 적용됩니다.

반영은 제어 명령급으로 봅니다 화면 구성을 바꾸는 것도 "시스템 판단에 영향을 주는 행위"라 감사 대상입니다. 다만 편집기가 아직 없으니 **자리만 만들어두고** 실제 기록은 나중에 시작합니다.

VZ-U-05 · 서브태스크 진행 상태 표시

한 줄로 말하면

"승인된 계획이 어디까지 갔고 어디서 실패했는지를 단계별로 보여줍니다."

쉽게 풀면 계획은 여러 구역에 걸쳐 **구간으로 쪼개져** 하달됩니다. 실패했을 때 **어느 구간에서 왜**인지는 화면에서만 알 수 있어요. 구간을 블록으로 늘어놓고 완료·실패·대기를 표시합니다.

왜 이 주기인가 상태가 바뀌는 건 하달·시작·완료·실패 네 시점뿐입니다. 수행 결과가 로봇 상태 데이터에 실려 오므로 별도 폴링이 필요 없어요.

VZ-U-06 · 미확인 탐지 그룹 확인

한 줄로 말하면

"AI가 '이게 뭘지 모르겠다'는 걸 모아서 보내면, 사람이 이름을 붙여 돌려줍니다."

쉽게 풀면 AI가 처음 보는 물체를 만나면 분류를 못 합니다. 그런 걸 **비슷한 것끼리 묶어서** 보내주면, 화면에서 "이건 소화 전이야"라고 알려주고, 그게 학습 후보로 반영됩니다.

중요한 원칙 사람 확인 없이는 학습에 확정되지 않습니다. 자동 라벨을 그냥 믿고 학습시키면 오류가 굳어져요.

VZ-U-07 · 계획 승인 및 근거 표시 (신설)

한 줄로 말하면

"AI가 만든 계획은 사람이 승인해야 실행됩니다. 그 판단 근거를 펼쳐서 보여줍니다."

쉽게 풀면 계획은 여러 단계를 거칩니다 — **전역 임무** → **구역별로 쪼개기** → **구간별 계획 생성** → **규칙 검증**. 검증을 통과 해도 **바로 실행되지 않습니다**. 사람이 승인해야 해요.

그러려면 화면이 **왜 이 계획인지**를 보여줘야 합니다. 어떤 임무에서 나왔고, 어느 구역을 어떤 순서로 지나고, 무슨 검증을 통과했고, 어떤 버전의 계획 생성기가 만들었는지까지요.

왜 필수인가 승인 없이 자동 실행하면 **책임소재가 성립하지 않습니다**. 사고가 났을 때 "AI가 했다"로 끝나버려요.

주의 이건 음성 명령과 무관합니다. **모든 계획**이 이 절차를 거칩니다. 음성은 그 입구 중 하나일 뿐이에요.

LLM · 음성 (VZ-L) — 4개

VZ-L-01 · 음성 입력(STT) 및 텍스트 전달

한 줄로 말하면

"말을 글자로 바꾸는 것까지가 제 몫이고, 그 뜻을 푸는 건 AI가 합니다."

쉽게 풀면 음성은 새로운 명령 체계가 아니라 입력 수단입니다. 마우스 클릭 대신 말로 하는 것뿐이에요. 그래서 기존 명령 경로를 그대로 탑니다. 백엔드 입장에서는 새 채널이 안 생깁니다.

"로봇 1번"을 어떻게 알아듣나 사람은 `go1-01` 이라고 말하지 않습니다. 그래서 레지스트리에서 **표시 이름과 별칭**을 같이 받아옵니다. 별도 시스템이 아니라 명부에 칸 하나 추가하는 정도예요.

왜 발화 끝날 때 한 번인가 계속 켜두고 들으면 관제실 잡음까지 전부 처리합니다. 오인식도 늘고 연산도 낭비고요.

VZ-L-02 · 해석 결과 표시 및 수락

한 줄로 말하면

"AI가 해석하고 만든 계획을 '이렇게 하면 될까요?'로 보여주고, 사람이 눌러야 실행됩니다."

쉽게 풀면 — 왜 해석을 AI에 넘겼나 해석기를 저와 AI가 각각 두면 **같은 말이 두 결과를 냅니다**. 그러면 문제가 생겼을 때 누구 탓인지 알 수 없어요. 해석과 계획 생성은 AI 한 곳으로 모으고, 저는 **전달하고 보여주고 수락받는** 역할만 합니다.

VZ-L-03 · 실행 전 2단 확인

한 줄로 말하면

"두 번 확인합니다 — 내가 말한 게 맞는지, 그리고 그걸 이렇게 해석한 게 맞는지."

쉽게 풀면 - 1차 — "제가 이렇게 들었는데 맞나요?" (잘못 들었을 수 있음) - **2차 —** "그 말을 이 임무·경로로 해석했는데 맞나요?" (잘못 해석했을 수 있음)

오류가 날 수 있는 지점이 두 군데니 확인도 두 번입니다.

되돌릴 수 있는 건 봐줍니다 화면 전환이나 조회는 잘못돼도 다시 누르면 그만이라 1차만 합니다. 하지만 **제어는 예외 없이** 두 번 거칩니다.

신뢰도가 낮으면 아예 AI에 보내지도 않고 다시 말해달라고 합니다. "높으면 건너뛰자"는 안이 있을 수 있는데, **사고는 애** 매한 구간에서 납니다.

VZ-L-04 · 화면 구성 보조 LLM

한 줄로 말하면

"제 쪽 LLM은 화면 만드는 걸 돕습니다. 로봇을 조종하는 해석은 안 합니다."

쉽게 풀면 "온도 높은 구역 보여줘" 같은 걸 화면 구성으로 바꾸거나, 데이터를 요약해주는 용도입니다. **로봇 제어 의도 해석은 AI 담당**이고 여기 포함되지 않습니다.

중요한 전제 LLM이 안 붙어 있어도 **나머지 기능은 전부 동작해야 합니다**. LLM은 있으면 좋은 것이지 없으면 안 되는 게 아닙니다.

공통 (VZ-C) — 2개

VZ-C-01 · 권한(RBAC)과 역할 확인

한 줄로 말하면

"화면에서 버튼을 가리는 건 편의고, 진짜 차단은 백엔드가 합니다."

쉽게 풀면 브라우저 코드는 사용자가 고칠 수 있습니다. 그러니 **화면에서 숨기는 건 방어가 아니에요**. 실제 검증은 백엔드가 명령을 받을 때 합니다. 저는 권한 없는 사람에게 안 보여주는 것까지만 합니다.

왜 로그인 때 한 번인가 역할은 세션 중에 거의 안 바뀝니다. 실제 방어선이 백엔드라 화면이 조금 늦게 알아도 위험하지 않고요.

VZ-C-02 · 외부 소스 부재 시 동작

한 줄로 말하면

"다른 파트가 아직 준비 안 됐어도 가시화는 혼자 돌아가야 합니다."

쉽게 풀면 전송 규격도, 레지스트리 형태도, 액션 어휘집도, 영상 변환 지점도 아직 확정 전입니다. 그게 정해질 때까지 **손 놓고 기다리면 시연 준비가 늦어요**. 그래서 정적 파일과 대체 데이터로 개발을 진행합니다.

이건 방어적 요구사항입니다 — "다른 파트 진척에 우리가 막히지 않는다"를 요구사항으로 못 박아둔 것.

지금 걸려 있는 것

다른 파트에 답이 걸려 있어 진행이 반쯤 묶인 항목입니다.

무엇	어디에 걸림	못 하는 것
좌표 기준·출처 필드	하드웨어 시트에서 빠짐	위치 신뢰도 표시(VZ-U-02)
프레임 참조 형식 통일	AI ↔ 하드웨어 형식 불일치	탐지 박스 겹치기(VZ-I-07)
명령 상관키	하드웨어 시트에 명시 없음	명령↔결과↔감사 연결
영상 변환 지점	미정	실시간 영상(VZ-I-06)
레지스트리 조회 형태	미정	미배포 표시(VZ-I-03)
백엔드 시트 전체	아직 비어 있음	수신 10건 중 5건의 근거

전부 대체 방법으로 개발은 진행 중입니다.

자주 나올 질문과 답

Q. 왜 다 백엔드를 거치나요? 직접 붙으면 빠를 텐데.

브라우저는 사용자 손 안에 있습니다. 접속 비밀번호를 브라우저에 내려주면 누구나 볼 수 있어요. 그리고 권한 제한이나 요청량 제한을 걸 곳도 백엔드밖에 없습니다.

Q. 상태를 왜 굳이 세 개로 나눠 받나요? 하나면 편한데.

"연결은 됐는데 기기가 고장난 상태"를 하나로 표현할 수 없습니다. 아는 주체가 셋이라(기기·서버·배포 시스템) 층도 셋이어야 합니다.

Q. 감사는 가시화가 쓰는 게 자연스럽지 않나요?

그러면 "누가 했다"를 클라이언트가 자기 입으로 말하는 구조가 됩니다. 조작할 수 있어요. 권한은 백엔드가 지키는데 기록만 브라우저가 쓰면 앞뒤가 안 맞습니다.

Q. 계획 승인을 화면에 두면 느려지지 않나요?

느려지는 게 목적입니다. 승인 없이 자동 실행하면 사고가 났을 때 책임소재가 성립하지 않아요. 대신 근거를 잘 보여줘서 승인 판단 자체는 빠르게 만드는 게 제 몫입니다.

Q. 주기 숫자는 어떻게 정했나요?

대부분 다른 파트가 정한 주기에서 따왔습니다. 원천이 15초에 한 번 주는 걸 1초마다 물어봐야 같은 값만 옵니다.

Q. 지금 못 하고 있는 게 있나요?

위의 "지금 걸려 있는 것" 표대로 여섯 가지가 다른 파트에 걸려 있습니다. 셋은 확인만 받으면 되고, 셋은 상대 파트의 결정이 필요합니다.